

AMATH 482: HOME WORK 2

JIAYUAN LI

Amath Department, University of Washington, Seattle, WA
jli72@uw.edu

ABSTRACT. This project applies principal component analysis (PCA) to motion capture data in order to extract low-dimensional structure from high-dimensional joint coordinate measurements. The dataset consists of walking, jumping, and running sequences recorded over time. PCA is used to identify dominant spatial modes and quantify the cumulative variance captured by leading components. The reduced representations are then used to perform classification via a nearest centroid method. Training and testing accuracies are evaluated as functions of the reduced dimension. The results demonstrate that a small number of principal components effectively captures the dominant motion dynamics while preserving classification performance.

1. INTRODUCTION AND OVERVIEW

High-dimensional datasets often contain substantial redundancy, with meaningful structure lying in a lower-dimensional subspace. Principal component analysis (PCA) is a classical dimensionality reduction technique based on the singular value decomposition (SVD) of a data matrix [3]. By identifying orthogonal directions of maximal variance, PCA provides a compact representation of complex data while preserving dominant patterns.

In this project, PCA is applied to motion capture data consisting of joint coordinates recorded over time for three types of human movement: walking, jumping, and running. Each time snapshot is treated as a spatial configuration in a high-dimensional space. The objective is twofold: first, to analyze the energy spectrum and determine how many principal components are required to capture the majority of variance; second, to evaluate whether the reduced representation enables reliable classification of motion types.

All computations were performed in Python using NumPy [1] and scikit-learn [4], with visualizations generated using Matplotlib [2].

2. THEORETICAL BACKGROUND

This project applies principal component analysis (PCA) to reduce the dimensionality of motion capture snapshots, followed by nearest centroid classification in the reduced space. PCA is derived from the singular value decomposition (SVD) and provides an optimal low-rank approximation of the data.

2.1. PCA via SVD. Let $x_i \in \mathbb{R}^d$ denote a snapshot and collect all centered training samples into

$$\tilde{X} \in \mathbb{R}^{n \times d}.$$

The economy SVD gives

$$\tilde{X} = U\Sigma V^T,$$

Date: February 12, 2026.

where singular values satisfy $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_r$. The principal directions are the columns of V . Projection into the k -dimensional PCA space is

$$z = (x - \mu)^T V_k.$$

2.2. Explained variance and truncation. The covariance matrix satisfies

$$\tilde{X}^T \tilde{X} = V \Sigma^2 V^T,$$

so the variance captured by component i is proportional to σ_i^2 . The cumulative energy ratio is

$$E_k = \frac{\sum_{i=1}^k \sigma_i^2}{\sum_{i=1}^r \sigma_i^2}.$$

In practice, k is chosen either by an energy threshold or by examining classification performance. PCA also yields the optimal rank- k approximation

$$\tilde{X}_k = U_k \Sigma_k V_k^T,$$

which minimizes $\|\tilde{X} - Y\|_F$ over all rank- k matrices Y . The reconstruction error equals $\sum_{i=k+1}^r \sigma_i^2$.

2.3. Nearest centroid classification. After projection, each class c is represented by its centroid

$$\bar{z}^{(c)} = \frac{1}{N_c} \sum_{i:y_i=c} z_i.$$

A new sample is classified by

$$\hat{y} = \arg \min_c \|z - \bar{z}^{(c)}\|_2.$$

This rule assigns labels based on Euclidean proximity in the reduced space.

3. ALGORITHM IMPLEMENTATION AND DEVELOPMENT

All computations were implemented in Python. Numerical array operations were performed using NumPy [1], principal component analysis was carried out using the scikit-learn implementation of PCA [4], and visualizations were generated using Matplotlib [2].

3.1. Data organization. Each motion sample is stored as a matrix

$$X \in \mathbb{R}^{114 \times 100},$$

where the first dimension corresponds to joint coordinates and the second dimension corresponds to time snapshots.

For PCA, the data were reorganized so that each time snapshot is treated as an individual observation in $\mathbb{R}^{114}$. For the training set, all snapshots were stacked to form a matrix of size

$$(15 \times 100) \times 114 = 1500 \times 114.$$

This matrix was used to fit the PCA model.

3.2. Principal component projection. PCA was fitted using only the training snapshots in order to avoid data leakage. After fitting, both training and testing samples were projected into the reduced k -dimensional PCA space.

For a fixed value of k , each sample of size 114×100 was transformed into a matrix of size $k \times 100$, preserving the temporal evolution of the motion in the reduced space.

3.3. Centroid computation. For each class (walking, jumping, running), centroids were computed in the k -dimensional PCA space using all projected training snapshots belonging to that class. That is, if z_i denotes a projected snapshot and y_i its corresponding label, the centroid for class c was defined as

$$\mu_c = \frac{1}{N_c} \sum_{i:y_i=c} z_i.$$

3.4. Nearest-centroid classification. Classification was performed at the snapshot level. For each projected snapshot z , the Euclidean distance to each class centroid was computed, and the predicted label was assigned according to

$$\hat{y} = \arg \min_c \|z - \mu_c\|_2.$$

Training accuracy was computed over all 1500 training snapshots, while testing accuracy was computed over all 300 testing snapshots. The procedure was repeated for varying values of k in order to study the trade-off between dimensionality reduction and classification performance.

4. COMPUTATIONAL RESULTS

In this section we present the numerical results of PCA-based dimensionality reduction and motion visualization for the captured humanoid joint data.

4.1. Task 1: Cumulative Energy and Mode Selection. We first examine the singular value spectrum of the centered data matrix and compute the cumulative energy captured by the first k PCA modes. The cumulative energy is defined as

$$E_k = \frac{\sum_{i=1}^k \sigma_i^2}{\sum_{i=1}^r \sigma_i^2},$$

where σ_i are the singular values of the centered data matrix.

Figure 1 shows the cumulative energy curve as a function of the number of PCA modes. We observe that the energy increases rapidly for small k and begins to plateau after only a few modes. In particular,

- $k = 2$ captures approximately 70% of the total energy,
- $k = 3$ captures approximately 80%,
- $k = 5$ captures approximately 90%,
- $k = 7$ captures approximately 95%.

This rapid decay of singular values indicates that the motion data lies near a low-dimensional manifold. Thus, only a small number of principal components are needed to retain most of the information.

4.2. Task 2: Visualization in Reduced PCA Space. We next project the training trajectories onto low-dimensional PCA subspaces in order to visualize the structure of the motion classes.

Figure 2a shows the projection onto the first two principal components. Even in two dimensions, the three motion classes (walking, jumping, and running) form visibly distinct clusters. Jumping appears concentrated in a relatively compact region, while walking and running occupy larger but still separated regions.

Figure 2b shows the projection onto the first three principal components. The additional dimension improves separation between walking and running trajectories, revealing clearer geometric structure in the data.

These results confirm that PCA effectively captures the dominant motion patterns and provides meaningful low-dimensional representations suitable for classification.

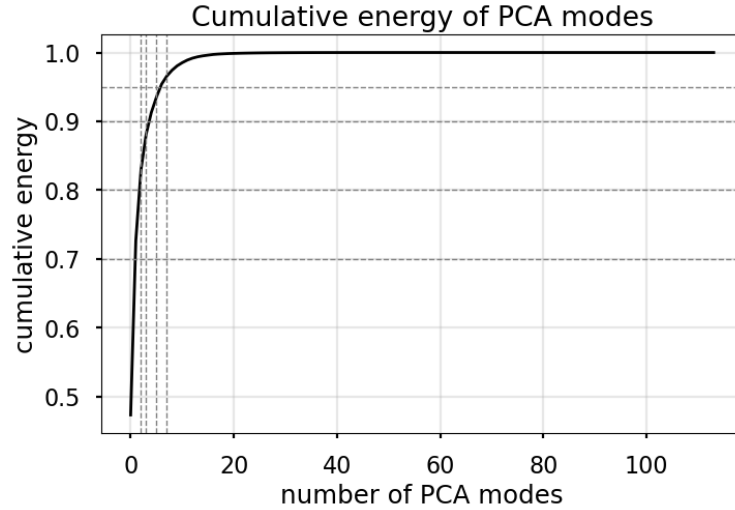


FIGURE 1. Cumulative energy captured by PCA modes. The dashed vertical lines indicate the number of modes required to retain common energy thresholds.

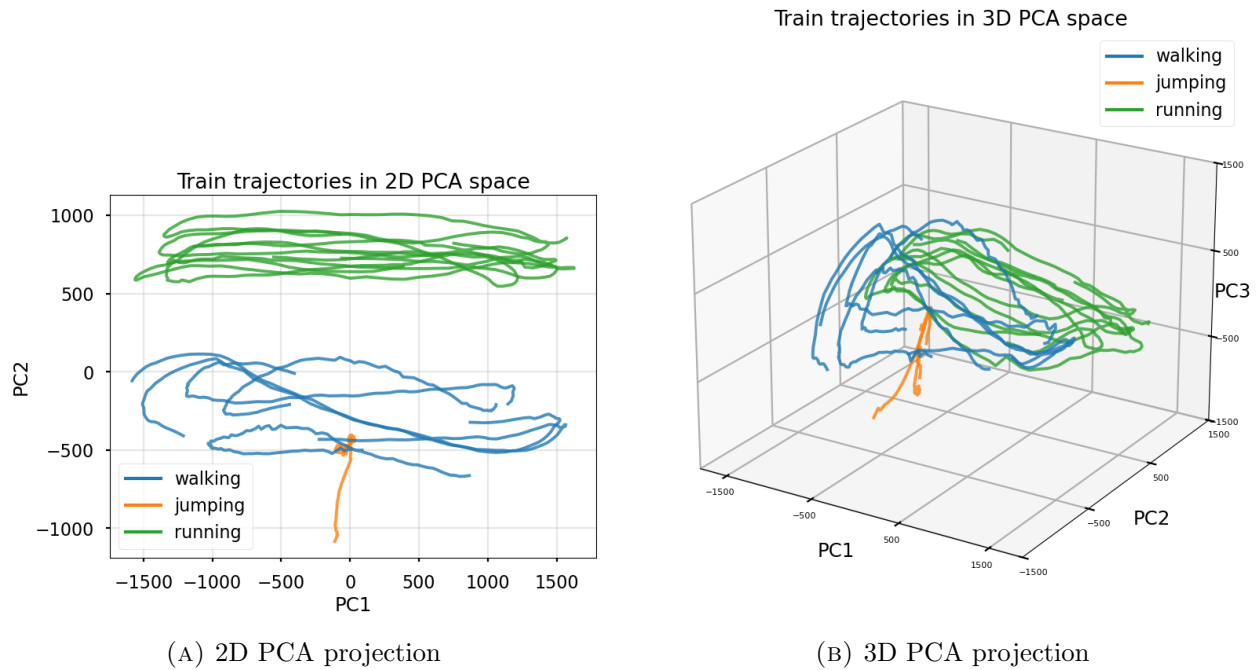


FIGURE 2. Training trajectories projected into reduced PCA spaces.

The strong class separation observed in 2D and 3D spaces further supports the rapid energy decay seen in Figure 1, indicating that most discriminative information is contained within only a few principal components.

4.3. Task 3: Snapshot-Level Centroids in k -PCA Space. For classification, we compute class centroids in k -dimensional PCA space using all time snapshots from the training set. Each centroid represents the mean location of all projected snapshots belonging to a given class.

This approach treats each time snapshot as an independent point in PCA space. As a result, classification is performed at the snapshot level rather than at the whole-trajectory level.

The computed centroids demonstrate that the three motion classes occupy distinct regions in PCA space, which motivates the use of a nearest-centroid classifier.

4.4. Task 4: Training Accuracy vs. k (Snapshot-Level). We evaluate the snapshot-level training accuracy for different values of k .

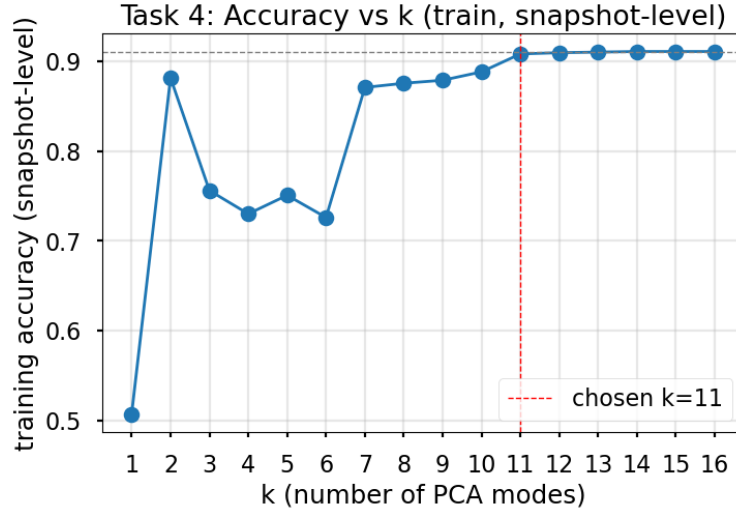


FIGURE 3. Training snapshot-level accuracy as a function of the number of PCA modes.

As shown in Figure 3, the accuracy increases significantly when moving from $k = 1$ to $k = 2$, indicating that the second principal component carries substantial discriminative information.

For larger k , the accuracy gradually stabilizes. Using the elbow rule, we select $k = 11$, where the training accuracy reaches approximately 90.8% and further increases in k yield only marginal improvement.

This reflects a balance between dimensionality reduction and classification performance.

4.5. Task 5: Test Accuracy vs. k . We evaluate the snapshot-level classification accuracy on the test set for different values of k . The results are summarized in Table 1.

k	1	2	3	4	5	6	7	8
Acc	0.487	0.983	0.923	0.747	0.917	0.717	0.943	0.930
k	9	10	11	12	13	14	15	16
Acc	0.943	0.943	0.953	0.953	0.953	0.953	0.953	0.953

TABLE 1. Snapshot-level test accuracy for different values of k .

We observe that although $k = 2$ achieves very high accuracy, the performance fluctuates for intermediate values of k . For $k \geq 11$, the test accuracy stabilizes near 95.3%, indicating that approximately 10–12 principal components are sufficient to capture the essential motion structure while maintaining strong generalization performance.

5. SUMMARY AND CONCLUSIONS

In this project, PCA was applied to motion capture data of walking, jumping, and running to investigate dimensionality reduction and classification performance. We analyzed the singular value spectrum and cumulative energy to determine suitable truncation levels. Low-dimensional projections (2D and 3D) revealed clear geometric separation between motion types in PCA space.

A nearest-centroid classifier was implemented in the reduced space, and accuracy was evaluated for various values of k . Results indicate that classification accuracy stabilizes for moderate values of k , suggesting that a relatively small number of principal components captures the dominant structure of the data. Overall, PCA provides both effective visualization and competitive classification performance with significant dimensionality reduction.

ACKNOWLEDGEMENTS

The author would like to thank Professor Natalie Frank for her insightful lectures on spectral methods, which provided the theoretical foundation for this work. Special thanks to the Teaching Assistant, Juan Ramirez, for helpful clarifications on Piazza regarding the implementation details of PCA and classification. The author also appreciates the discussions and collaborative support from lab partner Anna Chen.

REFERENCES

- [1] C. R. Harris, K. J. Millman, S. J. van der Walt, R. Gommers, P. Virtanen, D. Cournapeau, E. Wieser, J. Taylor, S. Berg, N. J. Smith, R. Kern, M. Picus, S. Hoyer, M. H. van Kerkwijk, M. Brett, A. Haldane, J. F. del Río, M. Wiebe, P. Peterson, P. Gérard-Marchant, K. Sheppard, T. Reddy, W. Weckesser, H. Abbasi, C. Gohlke, and T. E. Oliphant. Array programming with NumPy. *Nature*, 585(7825):357–362, Sept. 2020.
- [2] J. D. Hunter. Matplotlib: A 2d graphics environment. *Computing in Science & Engineering*, 9(3):90–95, 2007.
- [3] J. N. Kutz. *Data-driven modeling & scientific computation: methods for complex systems & big data*. OUP Oxford, 2013.
- [4] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.